

Two Answers to Per-Input Capacity: Sparse Mixture-of-Experts Routing versus a Held-Out Capacity Ladder

Jordan Elridge

10 July 2026

Summary

Two research communities have converged on the same question from opposite directions: *when a model is larger than any single input needs, how should each input decide how much of it to use?* In large language models the standard answer is the sparse **mixture-of-experts** layer: a learned gate picks, per token, a few “experts” out of many, so most of the model’s parameters sit idle on any one input. In our own work on probabilistic regression the answer is a **capacity ladder**: one nested model whose sub-models of every size are all valid, with the size each input deserves read afterwards from data the training never saw. The two designs solve the same problem — per-input activation of a subset of a large model’s capacity — with opposite methodologies, and this note makes the contrast precise by writing both as probability statements and auditing which of their ingredients follow from those statements.

The audit’s verdict, documented against the primary sources throughout: mainstream sparse mixture-of-experts training departs from a single probabilistic objective in identifiable, citable ways. Its gate is trained on the same objective as the experts, which is self-reinforcing — the authors of the founding system describe the favoured-expert feedback loop themselves — and the standard remedies are stabilisers added beside the likelihood: balancing losses with what their authors call “hand-tuned scaling factor[s]”, a router loss introduced explicitly as a numerical fix, a throughput cap that silently drops the tokens that exceed it, and gating noise whose role is to make one of the balancing losses differentiable. The known alternatives each *relocate* the arbitrary choice rather than remove it — one installs uniform load as an underived objective, one changes which side of the assignment chooses and pays for it elsewhere, one gives up discrete sparse routing altogether, and the newest replaces a tuned loss coefficient with a tuned bias step size that recent theory shows plays exactly the same role. The ladder framework, by contrast, keeps every training and selection ingredient a likelihood, a prior, or a proper-score operation on held-out data, and pays for that discipline elsewhere: in the cost of held-out charging at scale, and in measured limits of its own that this note reports with equal prominence. The contrast is methodological, not a performance claim: sparse mixture-of-experts routing is extraordinarily effective at what it was built for, and at its scale the trade it makes may well be the right one.

1. One problem, two methodologies

Fix a model whose total capacity exceeds what most single inputs need. Both frameworks must answer three questions. **Availability**: how is the model trained so that using only part of it is valid? **Selection**: who decides

how much of it a given input uses, and on what signal? **Deployment:** at prediction time, how is the decision executed cheaply? The mixture-of-experts answer trains the selector *inside* the training objective and executes it in the forward pass; the ladder answer trains only availability, moves all selection to *held-out data* — data set aside from training precisely so that scores on it are honest — and deploys the selection by distilling it into a small routing model afterwards. Throughout, we say a procedure **charges** for capacity when it makes extra capacity pay — score worse — unless the data genuinely support it; the note’s central question is where each framework’s charge comes from. Everything else unpacks the consequences of that one design difference.

Throughout, a **nat** is the natural-log unit of log-likelihood (one nat = one unit of $\log_e$ -probability), and a **proper scoring rule** is a score for probabilistic predictions whose expectation is maximised by the true distribution — so a model cannot game it except by being right. The **log score**, the log-probability a model assigns to the outcome that actually occurred, is the proper scoring rule used throughout.

2. The sparse mixture-of-experts layer as a probability statement

2.1 The full mixture

A mixture-of-experts layer holds E sub-networks (“experts”) $f_1, \dots, f_E$ and a gate $g(x) = (g_1(x), \dots, g_E(x))$, a set of non-negative weights summing to one, produced by a softmax over learned gate logits. Interpreted probabilistically, with each expert defining a predictive distribution $p_e(y | x)$ over the outcome y given the input x , the layer is a **conditional mixture**:

$$p(y | x) = \sum_{e=1}^E g_e(x) p_e(y | x), \quad (1)$$

the direct descendant of the adaptive-mixtures architecture of Jacobs et al. (1991), which was trained as exactly this likelihood — and, in its hierarchical form, fit by the expectation-maximisation algorithm, an exact coordinate ascent on that likelihood (Jordan and Jacobs 1994). This lineage is fully probabilistic: one objective, no extra terms. It is also *dense* — every expert runs on every input, which is precisely what modern systems cannot afford.

2.2 What top- k gating does to the statement

The modern sparse layer (Shazeer et al. 2017) keeps only the k largest gate weights per input (often $k = 1$ or 2 out of hundreds), sets the rest to zero, and renormalises the survivors. Writing $S_k(x)$ for the selected set, the computed object is the **truncated mixture**

$$\tilde{p}(y | x) = \sum_{e \in S_k(x)} \frac{g_e(x)}{\sum_{e' \in S_k(x)} g_{e'}(x)} p_e(y | x), \quad (2)$$

which is a perfectly valid k -component mixture *as a predictive object*. The departure is in training: the system trains **through** the truncation — gradients reach only the selected experts, and the selection itself is a non-differentiable top- k operation handled by training the gate on the truncated forward pass. So the object being optimised is not the mixture likelihood of Section 2.1, and the gate learns from a signal that only ever

includes the experts it already favours. Selection at inference and the training-time gradient path are different computational objects — a train/test mismatch that the probabilistic reading makes visible.

2.3 What the capacity factor does to it

Hardware adds a second departure. To keep the per-device workload bounded, each expert is given a fixed budget of inputs per batch — in the words of the Switch Transformer paper, “expert capacity = (tokens per batch / number of experts) $\times$ capacity factor” (Fedus, Zoph, and Shazeer 2022). (A word of warning: the “capacity” in this quoted term is a per-expert input budget, a hardware quantity — not the model-size sense of “capacity” used everywhere else in this note. We keep the term because it is the literature’s.) Inputs routed to a full expert are not processed by the layer at all: “If too many tokens are routed to an expert (referred to later as dropped tokens), computation is skipped and the token representation is passed directly to the next layer through the residual connection” (Fedus, Zoph, and Shazeer 2022). Probabilistically, the predictive distribution for a dropped input is replaced, silently and data-dependently, by whatever the surrounding network does with an unprocessed representation. No likelihood statement covers this; it is a throughput constraint acting directly on the model’s predictions.

2.4 The collapse, and the patches

Left alone, the in-objective gate fails in a characteristic way, described plainly by the authors of the founding system: “We have observed that the gating network tends to converge to a state where it always produces large weights for the same few experts. This imbalance is self-reinforcing, as the favored experts are trained more rapidly and thus are selected even more by the gating network” (Shazeer et al. 2017). Figure 1 illustrates the feedback loop on a two-expert toy (the dynamics are written out in Appendix B): the expert selected slightly more often improves slightly faster, which earns it more selection, and the gate rails to one expert. We have measured the same mechanism, at small scale, in our own companion work on probabilistic regression: a selection weight trained inside the training objective drifts to the largest capacity on offer or freezes onto a few components, because the training fit of extra capacity always grows while nothing inside the training objective charges for it. Figure 2 gives the schematic. The slope of its rising line is the classical price of one degree of freedom: each extra fitted parameter absorbs, on average, about one unit of squared standardised noise from the training data, and a Gaussian log-density pays for error through a $-\frac{1}{2}(\text{standardised error})^2$ term, so each absorbed unit is worth about half a nat of *training* fit. A gain that grows with every added parameter cannot be cancelled by any fixed in-training charge, so the charge must come from data the fit never saw.

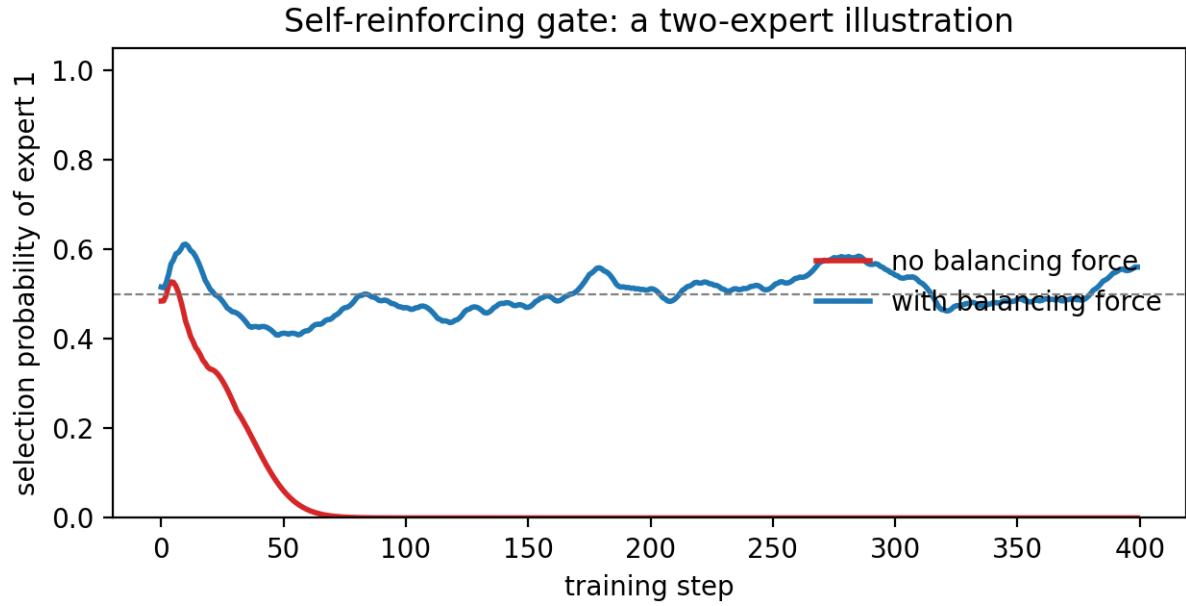


Figure 1: The self-reinforcing gate, illustrated on a two-expert toy (Appendix B gives the update rules and parameter values). Red: with no balancing force the gate rails to one expert — here the selection probability of expert 1 falls to zero as the other expert takes over. Blue: a balancing force holds the selection near even. This figure illustrates the mechanism; it is not a measurement of any production system.

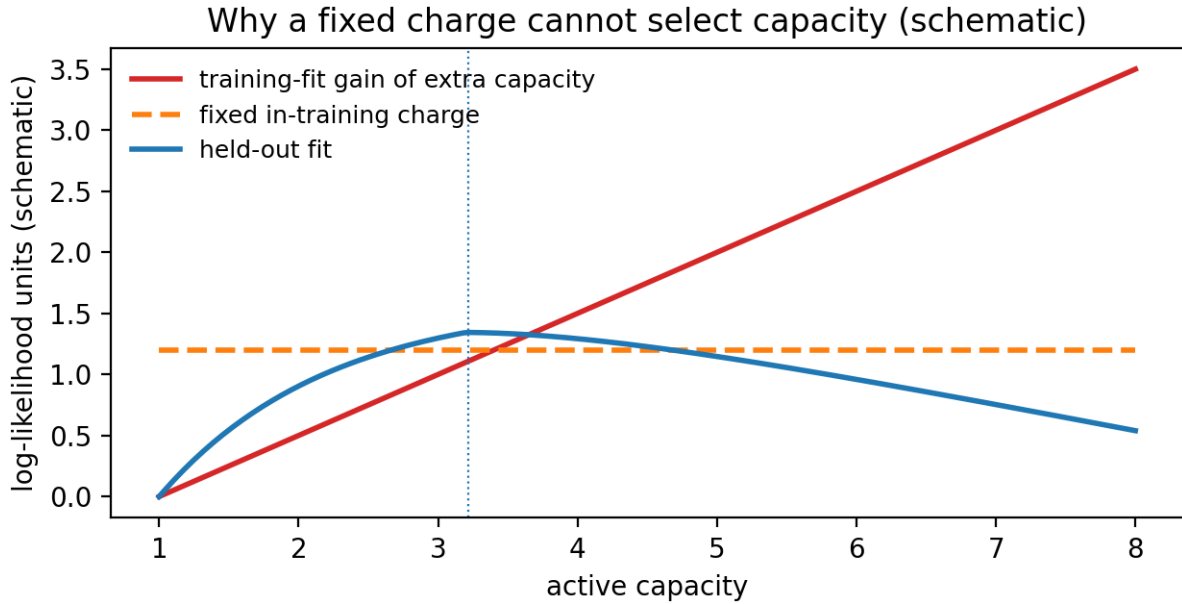


Figure 2: Why a fixed in-training charge cannot select capacity, schematically (active capacity measured in fitted parameters). The training-fit gain of extra capacity grows without bound — roughly half a nat per fitted parameter, by the degree-of-freedom accounting in the text; a fixed charge is flat, so it is overtaken at some capacity no matter where it is set; held-out fit peaks at the capacity the data supports and identifies it. This panel is a schematic of the mechanism, not a measured run.

The mainstream remedies are a family of stabilisers added *beside* the likelihood, each documented as such in its source:

- **The importance loss.** “We take a soft constraint approach,” write Shazeer et al. (2017), defining a penalty on uneven expert usage: “This loss is equal to the square of the coefficient of variation of the set of importance values, multiplied by a hand-tuned scaling factor $w_{\text{importance}}$.” (An expert’s *importance* is its total gate weight over a batch; the coefficient of variation is the standard deviation divided by the mean — a scale-free measure of unevenness.)
- **The load loss.** A second penalty of the same form — “the square of the coefficient of variation of the load vector, multiplied by a hand-tuned scaling factor w_{load} ” (Shazeer et al. 2017) — targeting the *number* of inputs each expert receives rather than its total gate weight. Their experiments set both hand-tuned factors to 0.1 in language modelling and 0.01 in translation.
- **Gating noise.** Random noise is added to the gate logits before the top- k cut; its stated role is that “the noise term helps with load balancing” — concretely, it turns each expert’s load into a smooth probabilistic quantity so the load loss has a usable gradient (Shazeer et al. 2017). Exploration noise in the service of differentiating a penalty.
- **The simplified balancing loss.** The Switch Transformer keeps one balancing loss with one coefficient, chosen empirically: “throughout this work we use an $\alpha = 10^{-2}$ which was sufficiently large to ensure load balancing while small enough to not to overwhelm the primary cross-entropy objective” (Fedus, Zoph, and Shazeer 2022).
- **The router z-loss.** A later stabiliser on the router’s raw logits, introduced with equal candour: “To fix this,

we introduce the router z-loss,” its weight “chose[n] ... based on the best model quality after pre-training with a hyperparameter sweep” ($c_z = 0.001$) (Zoph et al. 2022).

None of these terms is derived from the mixture likelihood, a prior, or any proper-score criterion; each exists to suppress a failure mode of training the gate in-objective, and each carries a coefficient set by search. That is not a hidden scandal — the authors say so themselves, and the quotations above are their own framing — but it fixes the methodological character of the recipe: the selector is stabilised by tuned side-objectives, not selected by evidence.

3. The capacity ladder as a probability statement

3.1 Availability by nesting

The ladder framework trains one model whose capacity settings are **nested**: setting c uses the first c of an ordered set of ingredients (the first c mixture components, the first c layers), and every prefix must be a valid model. We call each capacity setting a **rung** of the ladder. Training draws a capacity c at random for each example at each step and computes that example’s loss under the first c ingredients only — the loss itself is the ordinary negative log-likelihood of the renormalised sub-model. The draw distribution is a *schedule*: it decides which sub-models get gradient, and it adds no term to the objective. After training, every rung of the ladder is a trained model; nothing has been selected yet. This is the ordered-prefix training of Rippel, Gelbart, and Adams (2014), applied to a capacity axis.

3.2 Selection by held-out proper score

All selection then happens on a held-out sample, through one artifact: the table of per-example log scores $s_i(c) = \log p_c(y_i | x_i)$, the log-probability the capacity- c sub-model assigns to held-out example i ’s outcome. A single dataset-level answer comes from **stacking** (Yao et al. 2018): choose mixture weights over the rungs to maximise the held-out log score of the blend,

$$\hat{\pi} = \arg \max_{\pi \in \mathcal{S}} \sum_i \log \sum_c \pi_c e^{s_i(c)}, \quad \mathcal{S} = \left\{ \pi : \pi_c \geq 0, \sum_c \pi_c = 1 \right\}. \quad (3)$$

(Since $s_i(c) = \log p_c(y_i | x_i)$, the inner sum $\sum_c \pi_c e^{s_i(c)}$ is exactly the blend’s probability of held-out outcome i — the objective is the blend’s summed held-out log score.) Appendix A derives why this is the right selection objective — the log score is a proper scoring rule, so held-out-score-maximising weights are estimates of the best predictive blend, and the objective is concave, so the fit is exact — and how over-large rungs are charged automatically: they predict held-out data worse, so they earn near-zero weight with no penalty term anywhere. Per-input versions bin the held-out data by an observable feature and stack within bins (the input-dependent generalisation is hierarchical stacking, Yao et al. (2022)), or read a local contrast — the held-out advantage of the top rung over the bottom one, averaged over a neighbourhood of the input.

3.3 Deployment by distillation

The deployed router is trained *after* selection, by ordinary supervised likelihood, on targets measured from the held-out reads (the per-input soft weights the stacking produced). It is a small classifier fit to measured data —

nothing about it feeds back into the ladder’s training, so it cannot re-introduce the in-objective feedback loop. At prediction time it picks a rung per input and only that rung runs. In our companion study this distilled router matched or beat the single best global capacity on all nine test cases (for example, held-out negative log-likelihoods of 0.856 versus 0.885, 0.860 versus 0.949, and 0.826 versus 0.922 — router first, lower better — on the hardest problem’s three training runs) while carrying the per-input compute saving — and, notably, the router trained on *smooth held-out weights* beat the same router trained on hard per-input labels on seven of the nine cases, a small-scale echo of why soft, measured targets are safer selection signals than argmax decisions.

Every ingredient above is one of three things: a likelihood (the nested training loss, the router’s supervised fit), a prior (none was needed here, but a usage prior is the admissible slot — see Section 4), or a proper-score operation on held-out data (the stacking, the local contrast). That is the entire methodological claim; what it buys and what it costs are Sections 4–6.

4. The correspondence, term by term

The two frameworks line up ingredient by ingredient. The table gives the map; the prose after it walks the load-bearing rows.

Table 1: The structural correspondence. “Banned” rows are methodological choices of the ladder framework, not claims that the mixture-of-experts recipe could adopt them unchanged at its scale.

Mixture-of-experts ingredient	What it is, probabilistically	Ladder counterpart
Gate trained in-objective	selector fit on training data	banned: selection moved to held-out reads
Balancing losses (w, α : hand-tuned)	penalty beside the likelihood	banned: no tuned penalty anywhere
The balance goal itself, stated probabilistically	a prior, charged once	usage prior on dataset-average selection
Router z-loss (c_z : swept)	numerical patch on logits	not needed at this scale (no counterpart claim)
Capacity factor, token dropping	throughput cap overriding predictions	no counterpart: an honest gap (Section 6)
Noisy gating	noise making a penalty differentiable	capacity draws: a schedule, no objective term
Top- k train/test mismatch	trained and deployed objects differ	distilled router: deployed object fit directly
Expert-count choice (E, k)	fixed a priori	read from held-out data (stacking, contrast)

The gate and the selector. The self-reinforcing gate of Section 2.4 and the capacity selectors we measured failing in our own companion work are the same mechanism: a selector trained on the training objective, with no honest charge for what it selects, follows the training fit — and training fit always prefers more capacity for the favoured components and more gradient for the favoured experts. The mixture-of-experts community

patches the mechanism with balancing losses; the ladder framework removes the mechanism, by never training a selector in-objective at all.

The balancing losses and the prior. A hand-weighted penalty on uneven usage has a principled counterpart: a single prior on the *dataset-average* usage of the components, charged once in the objective — a Dirichlet prior (the standard distribution over weight vectors that sum to one) being the natural choice, because it turns the preference for balanced usage into one closed-form term in the average usage rather than a per-step penalty. The distinction matters: a prior is a fixed probabilistic statement whose strength does not race against a fit gain that grows with data and capacity, whereas a penalty coefficient must be re-tuned to stay in balance — which is exactly what the hand-tuned-factor quotations of Section 2.4 describe. (In our own experiments even the principled usage prior added nothing measurable once selection moved to held-out data; we record that as evidence about where the real work happens, not as an argument for the prior.)

Aux-loss-free balancing relocates the choice. The most recent mainstream development removes the balancing loss: DeepSeek’s method (Wang et al. 2024; DeepSeek-AI 2024) adjusts a per-expert bias added to the gate logits before the top- k cut, nudging under-used experts up and over-used experts down. Recent theory (Han and Zhong 2025) makes the accounting exact: the bias update “can be formulated as a primal-dual procedure that performs a single-shot update per iteration for finding a critical point of the Lagrangian” of a load-balance-constrained assignment problem. In plain terms: the balance requirement is enforced through a *price* per expert — the bias — raised while an expert is over-used and lowered while it is under-used, and the quoted result shows the bias update is exactly the textbook price-adjustment scheme for such a constraint. The step size u of that price adjustment is the new free knob. The paper’s own experiments search it over $u \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$. The arbitrary choice has moved from a loss coefficient to a price step size; it has not disappeared.

The other alternatives change the problem. The optimal-transport approach (BASE layers, Lewis et al. (2021)) removes the balancing loss by *assigning* inputs to experts under a hard uniform-load constraint — balance is achieved by installing uniform load as an objective in its own right, which no probability statement supplies. Expert-choice routing (Zhou et al. 2022) lets experts pick inputs instead of inputs picking experts, achieving balance by construction but letting each token’s compute depend on the other tokens in the batch — in autoregressive use this peeks at the future, and the published method needs its own patch for that. Soft mixing (Puigcerver et al. 2024) is genuinely free of balancing machinery, but it is not discrete sparse routing any more — every expert processes a weighted blend of inputs, a different computational object. And the fully principled ancestor — the expectation-maximisation mixture of experts (Jacobs et al. 1991; Jordan and Jacobs 1994) — is dense, which is the one thing a trillion-parameter layer cannot be. The pattern across all of them: for discrete sparse routing at scale, every known scheme either tunes a stabiliser, installs an underived objective, or changes the problem.

5. What the ladder buys, in evidence

The reason to hold the strictly probabilistic line is not aesthetics; it is that the removed mechanism was, in our companion study, the measured cause of wrong answers. Three results carry the point. First, in-objective selection failed in exactly the mixture-of-experts way: selection weights railed to the capacity cap or froze onto favoured components, on data whose true per-input capacity was known analytically. Second, held-out selection with the same models recovered the truth where a truth existed — the per-input component count on fixed-structure data, confirmed against known ground truth on every random seed — and, on the negative controls, *abstained*: single-component data, uniform-complexity data, and constant-noise data all read “no

extra capacity” (largest spurious per-input advantage at or below 0.016 nats, against 0.16–0.20 where structure is real), which is the behaviour that distinguishes an instrument from a structure-manufacturing heuristic. Third, the honest charge is levied with no tuning anywhere: over-large rungs earn near-zero stacking weight automatically, and rungs beyond what the data support show no held-out gain. The full tables and figures behind these headline numbers — including the framework’s own measured limits, which are real — are in the companion report (*Reading Model Capacity From Held-Out Data*, 2026, distributed alongside this note); this note deliberately reports the failures too (Section 6), because a methodological contrast drawn only from one side’s successes would be advocacy, not an audit.

6. The honest limits of the contrast

Scale. The mixture-of-experts recipe trains routers on the order of 10^{12} tokens inside the objective because that is nearly free there, whereas a held-out charge means scoring every candidate capacity on set-aside data. Our evidence is from studies of a few thousand points, where held-out scoring of a whole ladder costs one forward pass. At production scale the trade lands differently: the in-objective gate’s *systematic error* (bias in the statistical-estimator sense, not the logit offset of Section 4) may be cheaper than the held-out reader’s sampling noise and cost. The capacity-race arithmetic (Figure 2) still applies in principle — a fit gain that grows cannot be cancelled by a fixed charge at any scale — but where the practical optimum sits is an open empirical question this note’s evidence cannot settle.

Hardware. The capacity factor and token dropping answer a real constraint — bounded per-device memory and compute per batch — for which the ladder framework has no counterpart. A framework that has never had to schedule experts across accelerators has not earned an opinion on the machinery that does.

The ladder’s own measured limits. Held-out selection is not a free instrument. In our companion study: the single-answer selection rule built on the held-out scores (accept rungs while the held-out gain clears an error bar) mis-reads wherever the nested training’s own cost profile varies across rungs — any reader of the nested scores, the stacking weights included, inherits that distortion — and by exactly that mechanism it under-read the component count and over-read the network depth; the *coherence check* here is the measured held-out gap between each nested sub-model and a dedicated same-size model, and its profile across rungs is what leaks into the reads. The per-input read is power-limited, needing far more held-out data per input region than aggregate selection needs in total. And one structure (a component count that rises then falls across the input) was not recovered at all, plausibly because nesting imposes a single global importance ordering on components. (This does not conflict with the router result of Section 3.3: the router is scored on predictive fit, which can tie or win even where the underlying structure was not recovered.) Every one of these is a measured qualification of the “principled” side of this contrast, and any fair reading of the two methodologies should weigh them against the tuned stabilisers of the other.

What would change the verdict. The audit’s conclusion is about *derivability*, and it is falsifiable: a sparse-routing scheme whose balance emerges from a single likelihood-plus-prior objective with no swept constant — no loss coefficient, no dual step size, no installed uniform-load target — would close the gap this note documents. The theoretical framing of Han and Zhong (2025) is a step in that direction precisely because it makes the remaining free choice explicit rather than implicit.

Appendix A. Why held-out stacking is the right selection objective

We want the weights π over sub-models that make the blended prediction $p_\pi(y | x) = \sum_c \pi_c p_c(y | x)$ best for future data. “Best” needs a score, and the score must not be gameable: a **proper scoring rule** is one whose expected value, under the true distribution of outcomes, is maximised by predicting that true distribution. The log score $\log q(y)$ is strictly proper: for any candidate density q and true density p ,

$$\mathbb{E}_{y \sim p}[\log p(y)] - \mathbb{E}_{y \sim p}[\log q(y)] = \int p(y) \log \frac{p(y)}{q(y)} dy \geq 0, \quad (4)$$

with equality only at $q = p$ — the quantity on the right is the relative entropy, non-negative by Jensen’s inequality (the standard inequality that the average of a logarithm never exceeds the logarithm of the average). So maximising the *held-out* total log score of the blend over the N held-out examples,

$$\mathcal{L}(\pi) = \sum_i \log \sum_c \pi_c e^{s_i(c)}, \quad (5)$$

pushes p_π toward the data-generating distribution, using only data the sub-models never trained on — which is what levies the capacity charge: a rung that overfit its training data assigns low probability to held-out outcomes, its $e^{s_i(c)}$ terms are small, and any weight placed on it lowers $\mathcal{L}$. The optimisation is exact because $\mathcal{L}$ is concave in π : each term is the logarithm of a linear function of π , the logarithm is concave, and a sum of concave functions is concave. The maximiser is found by the fixed-point iteration

$$\pi_c \leftarrow \frac{1}{N} \sum_i r_{ic}, \quad r_{ic} = \frac{\pi_c e^{s_i(c)}}{\sum_{c'} \pi_{c'} e^{s_i(c')}}, \quad (6)$$

which sets each rung’s weight to the average share of held-out probability it carries. Each pass never decreases $\mathcal{L}$ — it is the expectation-maximisation update for a mixture-weight problem, and the standard argument applies: every pass maximises a lower bound on $\mathcal{L}$, obtained from the same Jensen’s inequality as above, that touches $\mathcal{L}$ at the current π — so the iteration climbs to the unique maximum of the concave objective. Nothing in this appendix contains a tunable constant.

Appendix B. The self-reinforcing gate, in two experts

The feedback loop quoted in Section 2.4 needs only two experts to appear. Let expert $i \in \{1, 2\}$ have an ability $a_i \geq 0$ (written a , not s — the $s_i(c)$ of Section 3 is an unrelated held-out score) that improves with practice and goes stale without it: when selected, $a_i \leftarrow a_i + \eta (1 - a_i)$ for a learning rate η , so ability rises toward a ceiling with use; when not selected, $a_i \leftarrow (1 - \rho) a_i$ for a small staleness rate ρ — the idle expert’s parameters fall behind as the rest of the network moves under it. Let the gate hold one logit difference z , select expert 1 with probability $\sigma(z) = 1 / (1 + e^{-z})$, and — mimicking gradient training on the shared objective — move toward whichever expert currently performs better: $z \leftarrow z + \eta (a_1 - a_2)$. Start symmetric, $a_1 = a_2 = 0$, $z = 0$ (Figure 1 uses $\eta = 0.25$, $\rho = 0.02$, and, for the balanced run, $\beta = 0.35$). The first few random selections break the tie: the expert selected slightly more often becomes slightly more able, the gate moves toward it, it is selected more, the neglected expert decays, and the loop closes — $\sigma(z)$ rails toward 0 or 1 (Figure 1, red; which expert wins is decided by the random early draws). Adding a balancing force $-\beta (\sigma(z) - \frac{1}{2})$ to the update holds the gate near even (Figure 1, blue) — which is precisely the role of the balancing losses of Section 2.4, and precisely why

removing them requires removing the in-objective gate itself. This two-expert system is an illustration of the mechanism, not a model of any production system: real routers have many experts, momentum, and shared layers, but the loop — selection begets ability begets selection — is the same one Shazeer et al. (2017) describe in words.

References

- DeepSeek-AI. 2024. “DeepSeek-V3 Technical Report.” *arXiv Preprint*.
- Fedus, William, Barret Zoph, and Noam Shazeer. 2022. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.” *Journal of Machine Learning Research* 23: 1–40.
- Han, X. Y., and Yuan Zhong. 2025. “A Theoretical Framework for Auxiliary-Loss-Free Load Balancing of Sparse Mixture-of-Experts in Large-Scale AI Models.” *arXiv Preprint*.
- Jacobs, Robert A., Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. “Adaptive Mixtures of Local Experts.” *Neural Computation* 3 (1): 79–87.
- Jordan, Michael I., and Robert A. Jacobs. 1994. “Hierarchical Mixtures of Experts and the EM Algorithm.” *Neural Computation* 6 (2): 181–214.
- Lewis, Mike, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. 2021. “BASE Layers: Simplifying Training of Large, Sparse Models.” In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 139:6265–74. PMLR.
- Puigcerver, Joan, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. 2024. “From Sparse to Soft Mixtures of Experts.” In *International Conference on Learning Representations (ICLR)*.
- Rippel, Oren, Michael A. Gelbart, and Ryan P. Adams. 2014. “Learning Ordered Representations with Nested Dropout.” In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 32:1746–54. PMLR.
- Shazeer, Noam, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.” In *International Conference on Learning Representations (ICLR)*.
- Wang, Lean, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. 2024. “Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts.” *arXiv Preprint*.
- Yao, Yuling, Gregor Pirš, Aki Vehtari, and Andrew Gelman. 2022. “Bayesian Hierarchical Stacking: Some Models Are (Somewhere) Useful.” *Bayesian Analysis* 17 (4): 1043–71.
- Yao, Yuling, Aki Vehtari, Daniel Simpson, and Andrew Gelman. 2018. “Using Stacking to Average Bayesian Predictive Distributions (with Discussion).” *Bayesian Analysis* 13 (3): 917–1007.
- Zhou, Yanqi, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. “Mixture-of-Experts with Expert Choice Routing.” In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- Zoph, Barret, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. “ST-MoE: Designing Stable and Transferable Sparse Expert Models.” *arXiv Preprint*.